Information Retrieval & NLP

Project 1 Report: TripAdvisor Recommendation System

Léo Winter and Alvaro Serero

ESILV A4 DIA6

Table of Contents

1. Introduction 3
2. Data & Preprocessing 3
1) Dataset Overview 3
2) Text Preprocessing 4
3) Dataset Preprocessing 4
3. Evaluation Methodology 4
1) Level 1 evaluation 4
2) Level 2 evaluation 4
4. Models 5
1) BM25 baseline model 5
2) TF-IDF model 5
3) Sentence-Transformers Model 6
4) Fusion Models 6
5. Results 6
1) Main Results 6
2) TF-IDF Ablation Study 7
6. Analysis 7
7. Conclusion 9

1. Introduction

The goal of this project is to build a content-based recommendation system for TripAdvisor places like hotels, restaurants, and attractions using only text from reviews written by users. Given the reviews of a query place, the system retrieves the most similar places from a candidate set. This system relies on the hypothesis that **similar experiences are described with similar words**.

No metadata like attraction category, cuisine type or price range is used as input to the model. Instead, these metadata fields serve exclusively as ground truth for evaluation, allowing us to measure whether the system successfully identifies places of the same type and subcategory.

2. Data & Preprocessing

1) Dataset Overview

In this project we have 7 datasets:

First, we have the **reviews** dataset that contains all the reviews along with their metadata. In this dataset, we only keep 3 columns:

- The *idplace* column that contains the id of the place which the line corresponds.
- The *review* column that contains the text of the review line
- The *langue* column that contains the language of the review

Second, we have the **trip_advisor** dataset that contains all the metadata for each place. In this dataset, we are only interested in 7 columns:

- The *id* column that contains the id of the place.
- The *typeR* column that contains the type of our place (H = Hotel | R = Restaurant | A = Attraction | AP = Attraction Product)
- *Then we have 5 columns that contains id: activiteSubCategorie, activiteSubType,* restaurantType, *restaurantTypeCuisine, priceRange*.

And then, we have 5 datasets acting as a dictionary to have the names corresponding to the id in a specific column in the **trip_advisor** dataset:

- The **AttractionSubCategorie** dataset for the *activiteSubCategorie* column
- The **AttractionSubType** dataset for the *activiteSubType* column
- The **cuisine** dataset for the *restaurantTypeCuisine* column
- The **dietary_restrictions** dataset for the *restaurantDietaryRestrictions* column
- The **restaurantType** dataset for the *restaurantType* column

In our project we will not use the **dietary_restrictions** dataset. To train our model we only used the **reviews** dataset.

2) Text Preprocessing

In the **reviews** dataset, we have around 45% of the reviews in English, 30% in French, and 25% in other languages. For this project, we only created a model for the English reviews. We have exactly 153,071 English reviews.

Then we used **spaCy** library to for the lemmatization (using the en_core_web_sm model) because spacy is the standard library for doing this for big volumes of data and is usually better than **ntlk** library.

- Since we added a lot of data to lemmatize, we decided to use the *pipe* function in spacy. It allows us to run a batch of reviews rather than one review by one review. It also allows us to disable the "parser" and "ner" module, as we did not need them, to reduce computation time.
- We also only kept alphabetic, non-stop tokens longer than 2 characters.

3) Dataset Preprocessing

First, we decided to group by our **reviews** dataset by place. For places that had more than 1 review, we created a list of at most 50 reviews.

Then, we merged the **trip advisor** dataset (only the 7 useful columns) with our new dataset to get the label of each place. That allows us to have everything we need to train and test our model in a single dataset that we named **reviews_place** containing 1,835 places distributed as follows: 989 AP, 538 R, 252 A, and 56 H.

To finish our preprocessing, we split our dataset into train and test sets with a separation of 50/50. For this split we used the *train_test_split* function from the **sklearn** library. We set the *shuffle* parameter to True to ensure the data was not ordered by any specific bias. And we used the *stratify* parameter on the **typeR** label to ensure that the original distribution of classes is strictly preserved in both the training and testing datasets. The split is deterministic (*random_state=42*) for reproducibility.

3. Evaluation Methodology

1) Level 1 evaluation

For level 1 evaluation, we check if the retrieved place is of the same type of the query place by checking the **typeR** attribute (H, R, A, or AP).

To obtain the error with level 1 evaluation we created a function named *ranking_error_L1:*

- We use the **tqdm** library to display the evaluation's progress.
- In case the model gives a type that doesn't exist we increase the undefined value.
- To obtain errors of a review, we search for the best rank that is of the correct type. We add this rank – 1 to the error list. The total error is the mean of all the errors.

2) Level 2 evaluation

For level 2 evaluation, we first check the type of the query place with the typeR attribute. Then:

- For attractions type: We check the *activiteSubCategorie* and *activiteSubType* attributes.
- For restaurants type: We check the *restaurantType and restaurantCuisine* attributes.
- For hotels type: We check the *priceRange* attribute.

To obtain the error with level 2 evaluation we created a function named *ranking_error_L2:*

- We use the **tqdm** library in the same way.
- In case the model gives a type that doesn't exist we increase the undefined value.
- In case the model gives a good type, but the sub type doesn't exist in our dataset then we increase the undefined value.
- To obtain errors of a review, we search for the best rank that is of the correct type and of the correct subtype. We added this rank – 1 to the error list. The total error is the mean of all the errors.

4. Models

1) BM25 baseline model

BM25 (Best Matching 25) is the standard baseline for information retrieval. It scores documents by summing IDF-weighted term frequencies with saturation and document-length normalization.

To implement this model, we used the *rank-bm25* Python library (BM25Okapi variant). BM25 is applied on the tokenized (split on whitespace) preprocessed documents.

2) TF-IDF model

Our main improved model uses TF-IDF vectorization with cosine similarity. While BM25 treats each query token independently, TF-IDF with cosine similarity captures the overall vocabulary profile of a place through the full vector representation. We chose this approach with three key parameters:

- **Unigrams + Bigrams (n-gram range 1–2):** Capturing two-word phrases such as "art museum", "wine list", or "rooftop terrace" allows the model to distinguish subcategories far more effectively than single words alone. The word "museum" appears across many attraction types, but "art museum" is highly specific.
- **Sublinear TF scaling (1 + log(tf)):** Without dampening, very frequent terms in long documents (generic words like "place", "visit", "nice" that survive stopword removal) dominate similarity scores. Sublinear scaling gives proportionally more weight to distinctive, less frequent terms.
- **Vocabulary size (50,000 features):** Our ablation study (Section 5) shows that larger vocabularies progressively improve performance, as they preserve rare but discriminative terms. We selected 50k as the final vocabulary size.

The vectorizer is fitted **only on training documents** so that the test vocabulary does not influence model construction.

3) Sentence-Transformers Model

To explore dense retrieval, we tested the *all-MiniLM-L6-v2* sentence transformer model, which produces 384-dimensional embeddings. Since this model has a **maximum context limit of 256 tokens**, encoding the full concatenated document would silently truncate most of the content.

To address this, we **encode each review individually**, then **average the embeddings per place** and re-normalize to unit length. Similarity is then computed via dot product (equivalent to cosine similarity for unit vectors). This strategy preserves information from all reviews rather than relying only on the first 256 tokens.

4) Fusion Models

Since sparse (BM25, TF-IDF) and dense (Sentence-Transformers) models capture complementary signals, we explored several combination strategies:

- **Linear Score Fusion (BM25 + TF-IDF):** We min-max normalize each model's scores to [0,1] per query, then combine them linearly:
  $score = \alpha \times BM25 + (1 - \alpha) \times TF_IDF$
  We swept α from 0.1 to 0.8 to find the optimal balance.
- **Reciprocal Rank Fusion (RRF):** RRF is a rank-based ensemble that does not require score normalization. For each candidate, it computes:
  $RRF(d) = \Sigma\,(1/(k + rank_i(d)))$, where k=60 is a smoothing constant. RRF is robust because it uses only ordinal positions, not raw scores, and has no hyperparameters to tune beyond k.
- **Three-Way Linear Fusion:** We also tested combining all three model scores (BM25, TF-IDF, and Sentence-Transformers) with various weight configurations, giving TF-IDF the largest weight since it dominates on L2 evaluation.

5. Results

1) Main Results

The table below summarizes the performance of all models :

Model	Error L1	Error L2
BM25 (baseline)	0.8364	5.2038
TF-IDF (unigrams + bigrams, sublinear, 50k)	0.71	3.1
Sentence-Transformers (per-review average)	1.2257	5.6930
Best Linear Fusion (BM25×0.1 + TF-IDF×0.9)	**~0.69**	**~3.074**
RRF (BM25 + TF-IDF)	~0.75	~3.55
RRF (BM25 + TF-IDF + ST)	~0.89	~3,81
Best 3-Way Fusion	~0.72	~3.21

The key takeaway is that **TF-IDF significantly outperforms BM25 at Level 2** (3.11 vs 5.20), reducing the ranking error by ~40%. At Level 1, improvement is more modest. Sentence-Transformers underperforms both sparse methods. Fusion and ensemble methods provide further incremental improvements.

2) TF-IDF Ablation Study

To justify our model design choices, we systematically varied TF-IDF hyperparameters. The table below shows the impact of each configuration:

TF-IDF Configuration	Error L1	Error L2	Δ vs best (L2)
unigrams, no sublinear, 10k	1.1363	5.6906	+2.59
unigrams, no sublinear, 20k	1.2028	5.8058	+2.70
unigrams, no sublinear, 50k	1.1908	5.7674	+2.66
unigrams, sublinear, 10k	0.9509	4.2182	+1.11
unigrams, sublinear, 20k	0.9945	4.4460	+1.34
unigrams, sublinear, 50k	1.0000	4.4388	+1.33
unigrams + bigrams, no sublinear, 10k	0.9487	4.1127	+1.01
unigrams + bigrams, no sublinear, 20k	1.0142	4.2446	+1.14
unigrams + bigrams, no sublinear, 50k	0.9498	4.8921	+1.79
unigrams + bigrams, sublinear, 10k	0.7874	3.2902	+0.18
unigrams + bigrams, sublinear, 20k	0.8375	3.2926	+0.18
unigrams + bigrams, sublinear, 50k	**0.7099**	**3.1055**	—

6. Analysis

1) Model Insights

- The **TF-IDF** model outperforms the BM25 model at both evaluation levels.

The custom-tuned TF-IDF configuration with bigrams, sublinear scaling, and vocabulary size captures better the specific linguistic patterns of our reviews than the more general purpose BM25.

- The **Sentence-Transformers** model underperforms compared to our other model. This is expected because:

The all-MiniLM-L6-v2 model is general purpose and is not fine-tuned on review text. Also, the project hypothesis is that similar places use similar words. Our TF-IDF model directly measures word overlap, while dense embeddings capture more abstract semantics that compress away the discriminative vocabulary

- **Fusion methods** provide marginal improvements over TF-IDF:
  - The *Linear fusion* of BM25 and TF-IDF with low BM25 weight slightly improves L2 and L1 score. Our best L1 and L2 scores are with this method with $\alpha = 0.2$.
  - *Reciprocal Rank Fusion* is competitive and requires no hyperparameter tuning for score normalization. But these models are worse in L1 and L2 compared to our fine-tuned TF-IDF model or other fusion methods.
  - *Three-way fusion* (including Sentence-Transformers) can further improve results, showing that dense embeddings still contribute complementary information. But these models are still not as good as a good linear fusion model or a fine-tuned TF-IDF model for L1 and L2 evaluations.

2) Ablation Study Insights

- **n-gram** parameters

We can see that having unigram and bigram allows us to have lower L1 and L2 error compared to only having unigram.

- **Sublinear** parameters

We can see that if we have the sublinear parameters set to true, we have large improvements to our L1 and L2 errors.

- **Vocabulary size** parameters

We can see that the best vocabulary size depends on other parameters, and 50k is not always the best parameter.

3) Limitations

Several limitations affect our evaluation.

- First, the high number of undefined queries at L2 (~500/917) is due to missing subcategory metadata for many Attraction Products.
- Second, the hotel category has only 56 places (28 per split), making its error estimates noisy.
- Third, we only use English reviews (45% of corpus); incorporating French reviews (29%) through translation or multilingual models could improve coverage.

7. Conclusion

Our results **validate the project hypothesis**: similar places are indeed described with similar words, and text-based retrieval can effectively recommend places of the same type and subcategory without any metadata. The system achieves L1 errors below 1.0 (the first retrieved place is almost always the correct type) and L2 errors around 3 (the correct subcategory is found within the top few results).

The best single model is TF-IDF with bigrams, sublinear TF, and 50k vocabulary, which reduces BM25's L2 error by ~40%. Ensemble methods (particularly Reciprocal Rank Fusion combining all three models) provide incremental improvement, but aren't superior. The systematic ablation study confirms that each design choice (bigrams, sublinear scaling, vocabulary size) contributes meaningfully to the final performance.

All code is provided in the accompanying Jupyter notebook, which can be run end-to-end to reproduce all results presented in this report.